

Logical Operators

04 July 2026 18:31

There are four logical operators in Javascript : `||(Or)`, `&&(And)`, `!(Not)`, `??(Nullish Coalescing)`.

Although they are called "logical". They can be applied to values of any type, not only boolean. The result can also be of any type.

||(OR): The OR operator is represented with two vertical lines symbols:

```
result = a || b
```

In classical programming, the logical OR is meant to manipulate boolean values only. If any of its arguments are true, it returns true, otherwise it returns false.

In Javascript the operator is little bit trickier and more powerful. But first, let's see what happens with boolean values.

There are four possible logical combinations:

```
console.table(true || true); //true
console.table(false || true); //true
console.table(true || false); //true
console.table(false || false); //false
```

Output:

```
true
true
true
false
```

As we can see, the result is always true, except for the case when both operands are false.

If an operand is not a boolean. It's converted to a boolean for the evaluation.

For instance, the number one is treated as true, the number zero as false:

```
console.table(true || true); //true
console.table(false || true); //true
console.table(true || false); //true
console.table(false || false); //false
if(1 || 0){
  console.table('truthy!')
}
```

Output:

```
true
true
true
false
truthy!
```

Most of the time Or `||` is used in an if statement to test if any of the given condition is true.

```
const prompt = require('prompt-sync')();
let hour = Number(prompt('Enter hour(0-24): '));
let weekDay = prompt('Enter name of week day(Sun, Mon,...Sat): ');
weekDay = weekDay.toLowerCase();
let isWeekend = (weekDay === 'sat' || weekDay === 'sun');
if(hour < 10 || hour > 18 || isWeekend){
    console.log('The office is closed.');
```

Example:

```
Enter hour(0-24): 16
Enter name of week day(Sun, Mon,...Sat): Fri
The office is open.
```

OR "||" finds the first truthy value

The logic described above is somewhat classical. Now let's bring in the extra features of Javascript. The extended algorithm works as follows.

Given multiple OR'ed value:

```
result = value1 || value2 || value3;
```

The OR `"||"` operator does the following:

- Evaluates operands from left to right.
- For each operand converts it to boolean. If the result is true, stops and returns the original value of that operand.
- If all operand have been evaluated(i.e. all were false), returns the last operand.

A value is returned in its original form without the conversion.

In other words, a chain of or returns the first truthy value for the last one if no truthy value is found.

More examples:

```
console.log(1||0); //1 (1 is truthy)
console.log( null || 1); //1 (1 is the first truthy value)
console.log( null || 1 || 0); //1 (The first Truthy value)
```

```
console.log( undefined || null || 0); // 0 (all falsy, returns the last value)
```

Output:

```
1
1
1
0
```

This leads to some interesting usage compared to a "pure classical boolean-only OR".